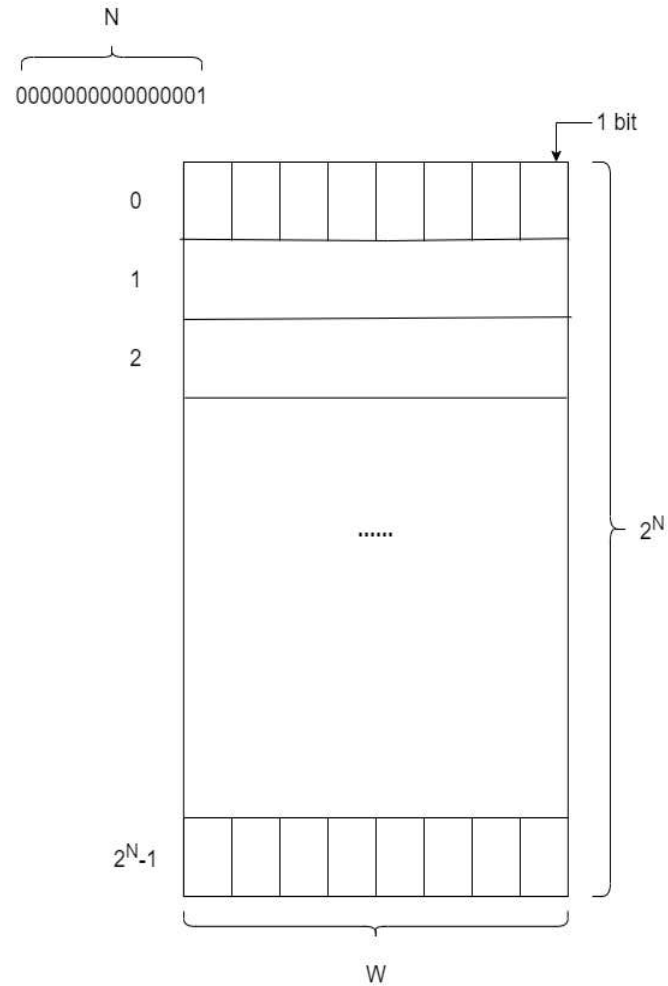


Syntax of pointers

Declaration and Initialization

Review: memory

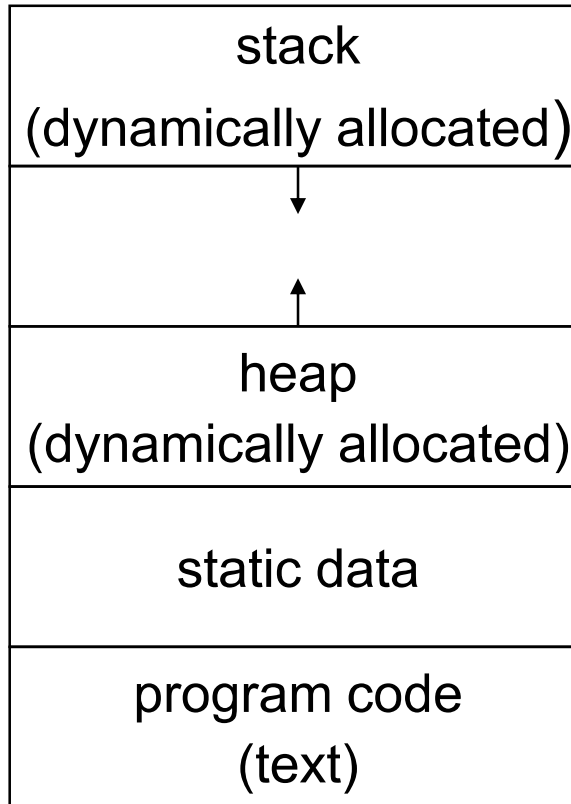


Memory layout of C programs

0xFFFFFFFF

address
space

0x00000000



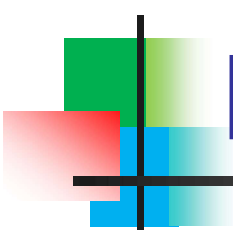
← SP

← PC



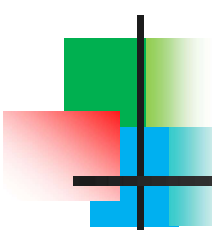
Definition – *Pointer*

- A *value* indicating the *number* of (the first byte of) a data object
 - Also called an *Address* or a *Location*
- Used in machine language to identify which data to access
- Usually 1, 2, 4, or **8 bytes**, depending upon machine architecture



Pointers in C

- **'&'** unary “address of” operator
 - generates a *pointer* to **x**
 - E.g., **scanf("%d", &x);**
 - E.g., **p = &c;**
 - Operand of **'&'** must be an *l-value*
- Unary **'*'** operator *dereferences* a pointer
 - i.e., gets value pointed to
 - E.g. ., **printf("%d",*p)** refers to value of **c** (above)
 - Can also use as lvalue,
 - E.g., ***p = x + y; *p = *q;**



Declaring Pointers in C

- `int *p;` — a pointer to an **int**
- `double *q;` — a pointer to a **double**
- `char **r;` — a pointer to a pointer to a **char**
- `type *s;` — a pointer to an object of
type *type*
 - E.g, a **struct**, **union**, **function**, something
defined by a **typedef**, etc.

Operator Precedence and Associativity

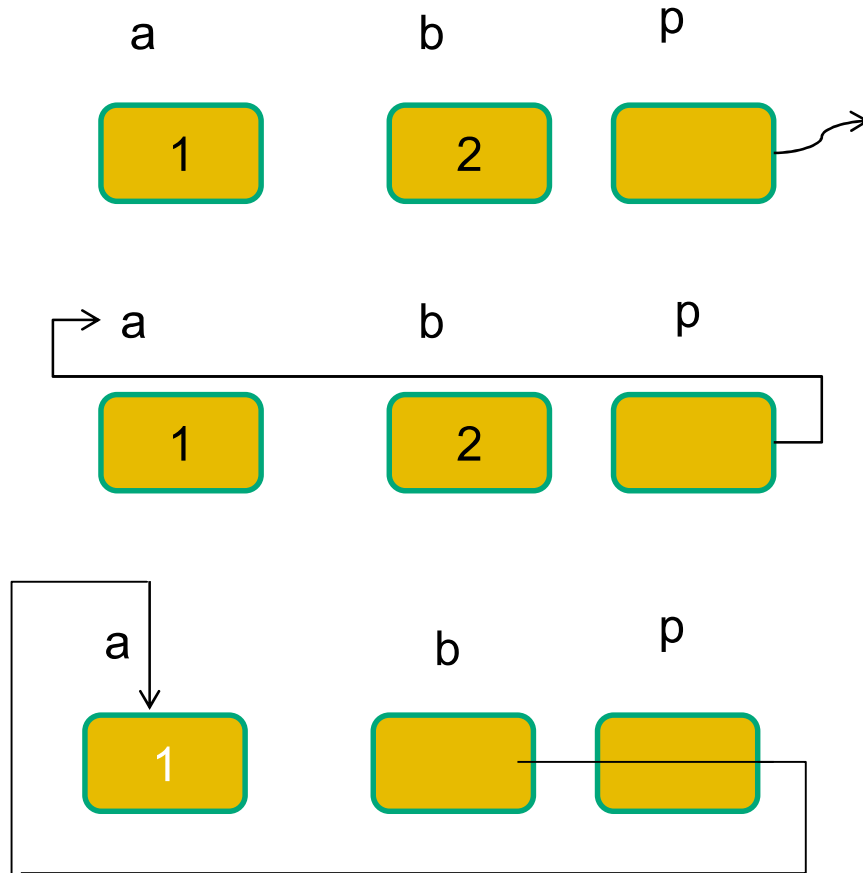
Operator	Associativity
() [] ++(postfix) --(postfix)	left to right
+(unary) -(unary) ++(prefix) --(prefix) ! (type) & *	right to left
* (multiplication) / %	left to right
+ -	left to right
< > <= >=	left to right
== !=	left to right
& (bitwise AND)	left to right
(bitwise OR)	left to right
&& (logical AND)	left to right
(logical OR)	left to right
?:	right to left
= += -= *= /= etc	right to left
, (comma operator)	left to right

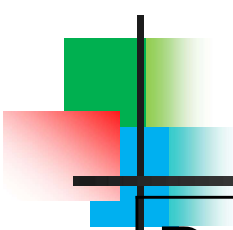
Example

```
int a = 1, b = 2, *p;
```

```
p = &a;
```

```
b = *p;
```

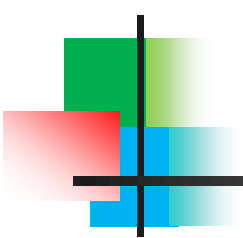


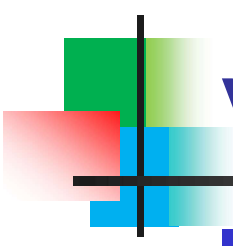


Declarations and Initializations

```
int i = 3, j = 5, *p = &i, *q = &j, *r;  
double x;
```

Expression	Equivalent Exp.	Value
<code>p == &i</code>	<code>p == (&i)</code>	1
<code>**&p</code>	<code>*(&p)</code>	3
<code>r = &x</code>	illegal	
<code>7 ** p / *q + 7</code>	<code>((7 * (* p)) / (*q)) + 7</code>	11
<code>* (r = &j) ** * p</code>	<code>(* (r = (&j))) ** (* p)</code>	15

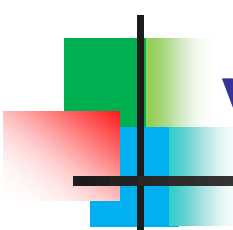
- 
-
1. Write a C program to read two numbers from user and add, subtract, multiply and divide them using pointers.



void pointer

- Pointer to an object of unknown data type
- Generic pointer with no data type associated

`void *pointer`



void pointer

- Memory required for void pointer is 8 bytes

```
void *ptr;
```

```
char cnum;
```

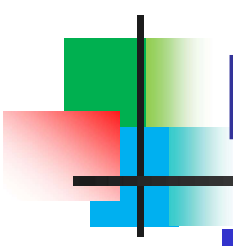
```
int inum;
```

```
float fnum;
```

```
ptr = &cnum;
```

```
ptr = &inum;
```

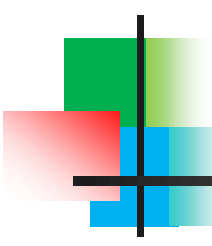
```
ptr = &fnum;
```



Dereferencing void pointer

- Compiler has no knowledge what type of object a void pointer points to

```
char cnum = 'a';  
void *ptr;  
ptr = &cnum;  
printf("%c",*ptr); // Compiler warning/error  
_____ // type casting required
```



Array of void pointer

```
int num1 = 1, num2 = 2, num3 = 3;
```

```
void *ptr[3];
```

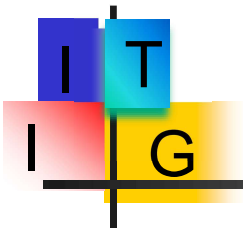
```
ptr[0] = &num1;
```

```
ptr[1] = &num2;
```

```
ptr[2] = &num3;
```

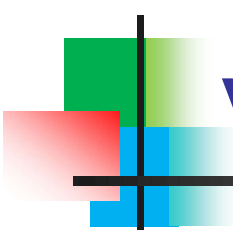
```
for (int i = 0; i < 3; i++)
```

```
    printf("%d\t", *(int *)ptr[i]);
```



Syntax of pointers

Constant qualifier



void function pointer

Write a function to find the maximum of two numbers using void pointers.

- The function prototype is

`void* max(void *a, void *b, char type)`

where type specifies data type ('i' for int, 'f' for float)

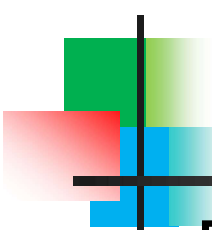
- Use the ?: operator



const Qualifier

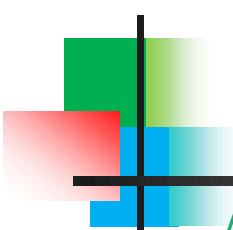
const qualifier

- Variable cannot be changed
- Use const if function does not need to change a variable
- Attempting to change a const variable produces an error
- `const float PI = 3.14; // This will define PI as constant with value 3.14`



const Qualifier and pointers

- Pointer declarations:—read from *right* to *left*
- **const float *ptr;**
 - `ptr` is a pointer to a floating point constant
 - i.e., pointer can change, thing it points to cannot
- **float * const ptr;**
 - `ptr` is a constant pointer to a floating point variable
 - i.e., pointer cannot change, thing it points to can!
- **const float * const ptr;**
 - `ptr` is a constant pointer to an integer constant
- Constant variables/pointers must be initialized when defined



float * ptr;

ptr is a pointer to float

const float * ptr;

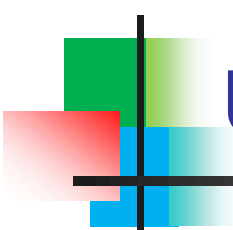
ptr is a pointer to a float constant

float * **const** ptr;

ptr is a constant pointer to float

const float * **const** ptr;

ptr is a constant pointer to
a float point constant

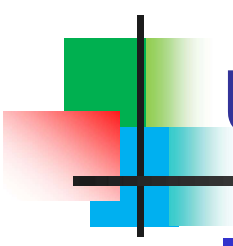


use cases of `const` qualifier

- What will be the output?

```
#include<stdio.h>
```

```
int main(){  
    const float PI = 3.14;  
    const float *ptr1 = &PI;  
    *ptr1=3.1416;  
}
```

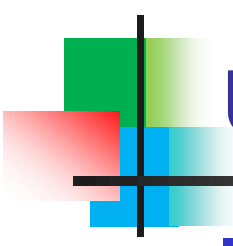


use cases of `const` qualifier

- Pointer to constant
- What will be the output?

```
#include<stdio.h>
```

```
int main(){  
    const float PI = 3.14;  
    float *ptr1 = &PI;  
    *ptr1=3.1416;  
    printf("%f\n", *ptr1);  
}
```

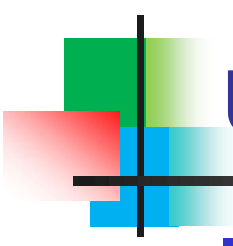


use cases of const qualifier

- Constant pointer
- What will be the output?

```
#include<stdio.h>
```

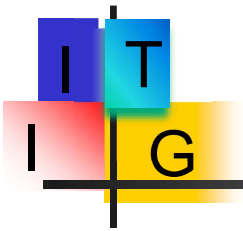
```
int main(){  
    float PI = 3.14, num1;  
    float *const ptr1 = &PI;  
    ptr1 = &num1;  
}
```



use cases of `const` qualifier

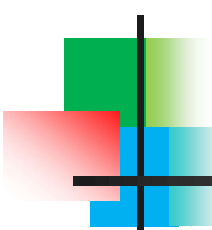
- Constant pointer to constant
- What will be the output?

```
#include<stdio.h>
int main(){
    float PI = 3.14, num1;
    const float *const ptr1 = &PI;
    *ptr1 = 3.1416;
    ptr1 = &num1;
}
```



Pointers and functions

Call by reference



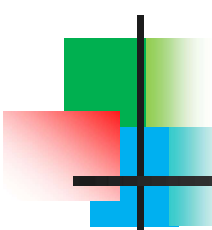
Calling Functions by Reference

- Function definition

```
return-type function-name( type *para1, type *para2.. )  
{  
  declarations and statements  
  // Use dereferenced pointer  
}
```

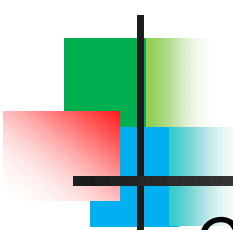
- Calling function

```
function-name( &arg1, &arg2.. )
```



Calling Functions by Reference

- Call by reference with pointer arguments
 - Pass address of argument using **&** operator
 - Allows you to change actual location in memory
 - Arrays are not passed with **&** because the array name is already a pointer
- Use dereference pointer in the function body



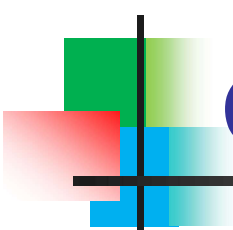
swap() function revisited

- Call-by-value: what went wrong?

```
void swap(int p, int q) {  
    int tmp;  
    tmp=p; p=q; q=tmp; }
```

```
int main(void) {  
    int i = 3, j = 5;  
    swap(i,j);  
    printf("Value of i=%d,j=%d\n",i,j);  
}
```

C passes arguments by value, which means the function receives copies of the variables, not the originals.



Call-by-value

```
int main(void) {  
    ...  
    swap(i,j);  
.. }
```

```
void swap(int p, int q) {  
    int tmp;  
    tmp=p; p=q; q=tmp; }
```

p

i

q

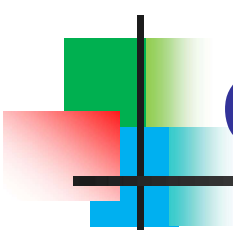
j

3

3

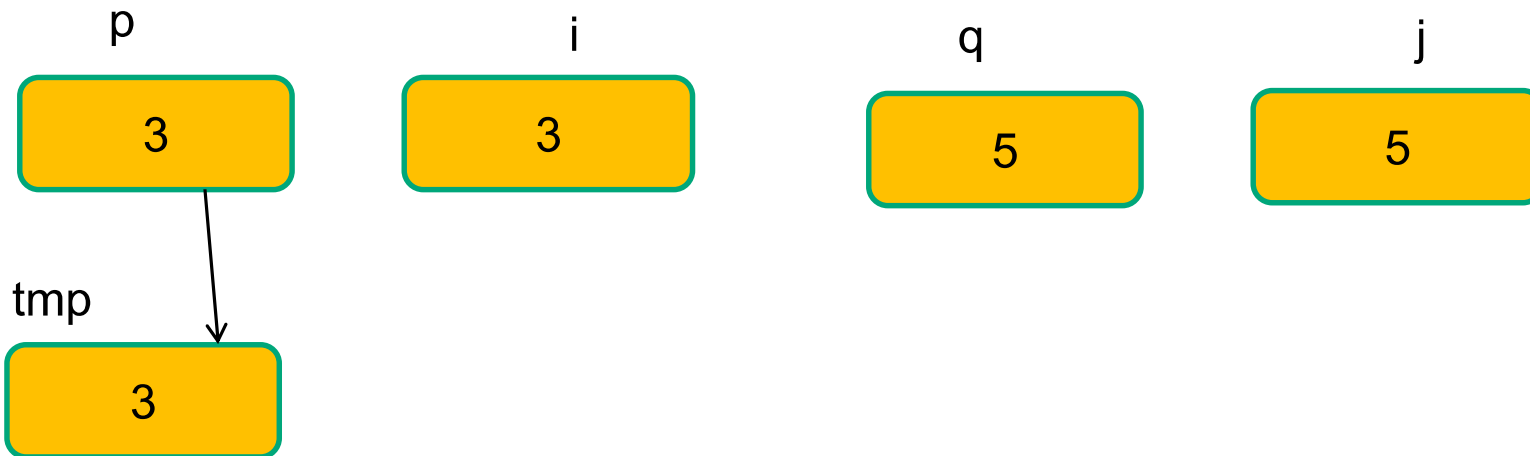
5

5



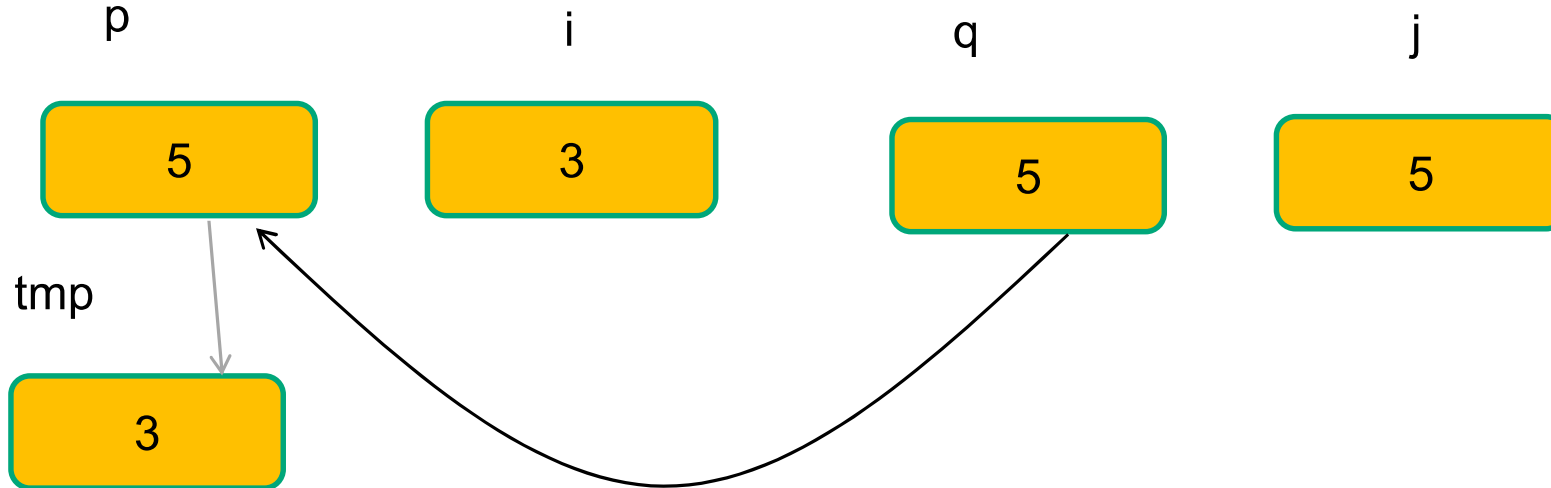
Call-by-value

```
void swap(int p, int q) {  
    int tmp;  
    tmp=p; p=q; q=tmp;  
}
```



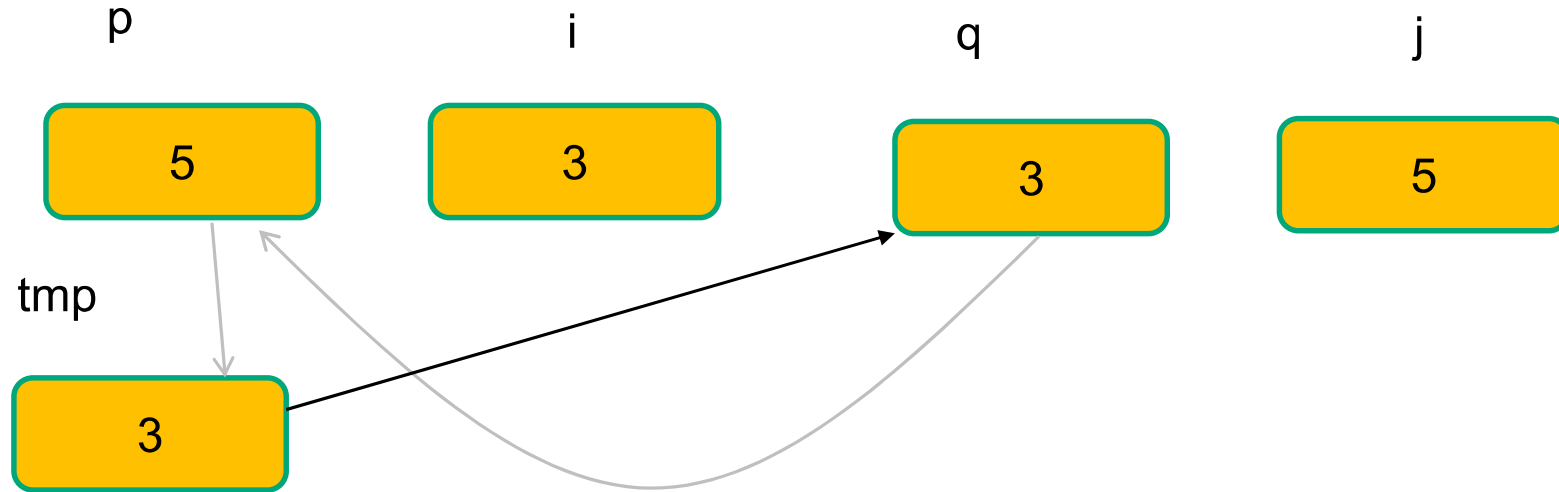
Call-by-value

```
void swap(int p, int q) {  
    int tmp;  
    tmp=p; p=q; q=tmp;  
}
```



Call-by-value

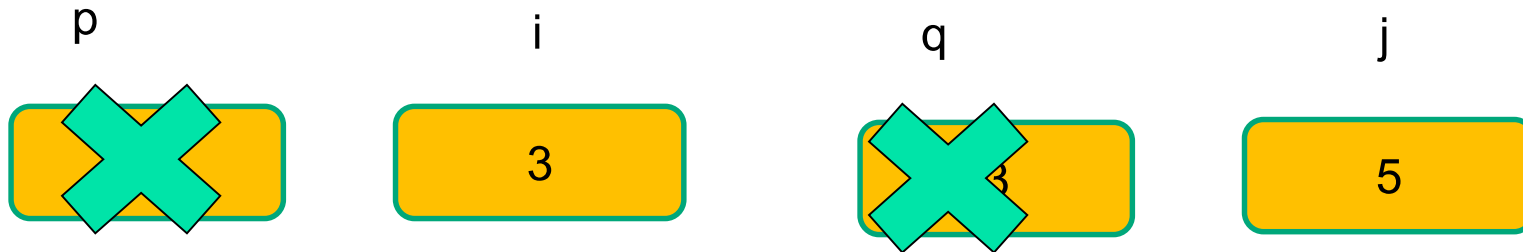
```
void swap(int p, int q) {  
    int tmp;  
    tmp=p; p=q; q=tmp;  
}
```

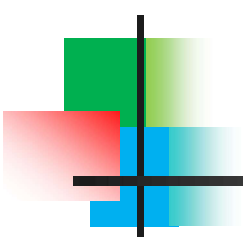


swap() function – what went wrong?

```
int main(void) {  
    int i = 3, j = 5;  
    swap(i,j);  
    printf("Value of i=%d,j=%d\n",i,j);  
}
```

```
void swap(int p, int q) {  
    int tmp;  
    tmp=p; p=q; q=tmp; }  
}
```

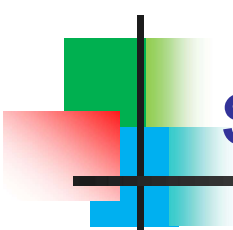




swap() function using call by reference

```
int main(void) {  
    int i = 3, j = 5;  
    swap(i,j);  
    printf("Value of i=%d,j=%d\n",i,j);  
}
```

```
void swap(int p, int q) {  
    int tmp;  
    tmp=p; p=q; q=tmp; }
```



swap() using Call-by-Reference

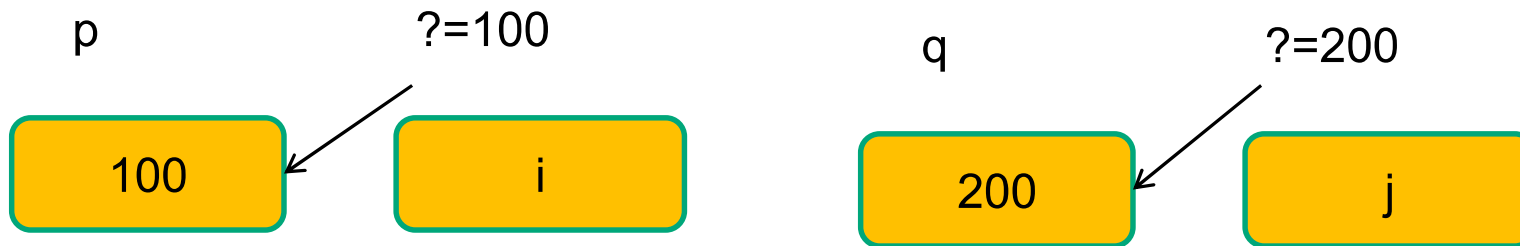
```
int main(void) {  
    int i = 3, j = 5;  
    swap(&i,&j);  
    printf("Value of i=%d,j=%d\n",i,j);  
}
```

```
void swap(int *p, int *q) {  
    int tmp;  
    tmp=*p; *p=*q; *q=tmp; }
```

Call-by-Reference

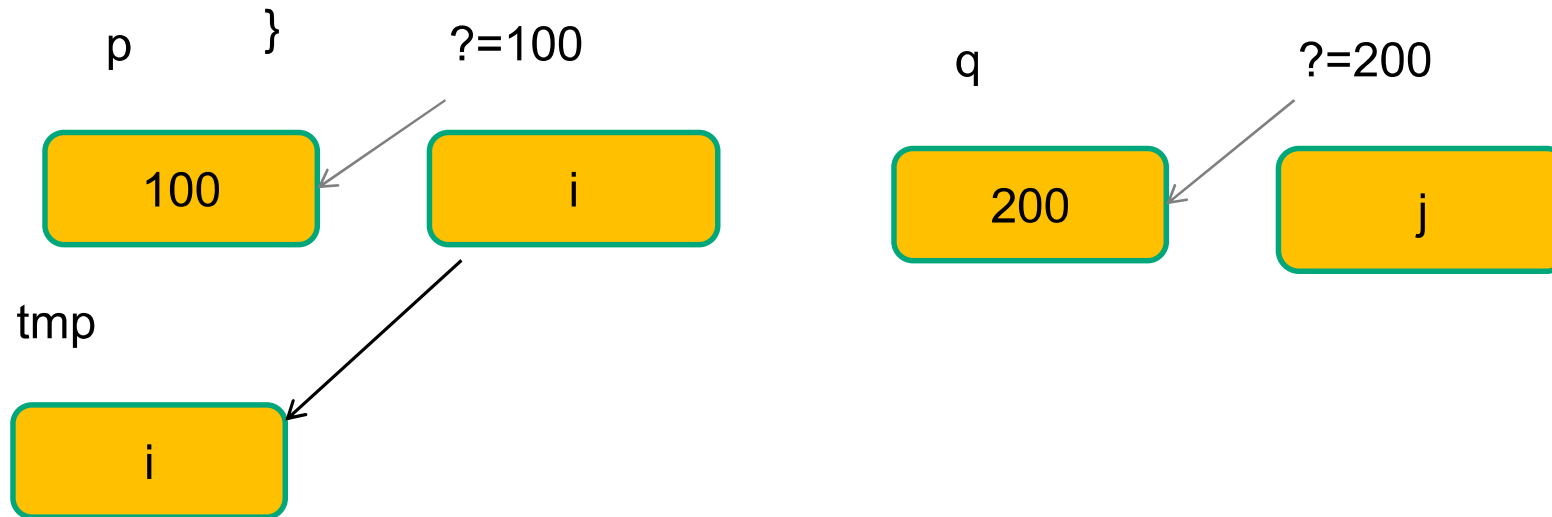
```
int main(void) {  
    ...  
    swap(&i,&j);  
    .. }  
.. }
```

```
void swap(int *p, int *q) {  
    int tmp;  
    tmp=*p; *p=*q; *q=tmp; }  
.. }
```



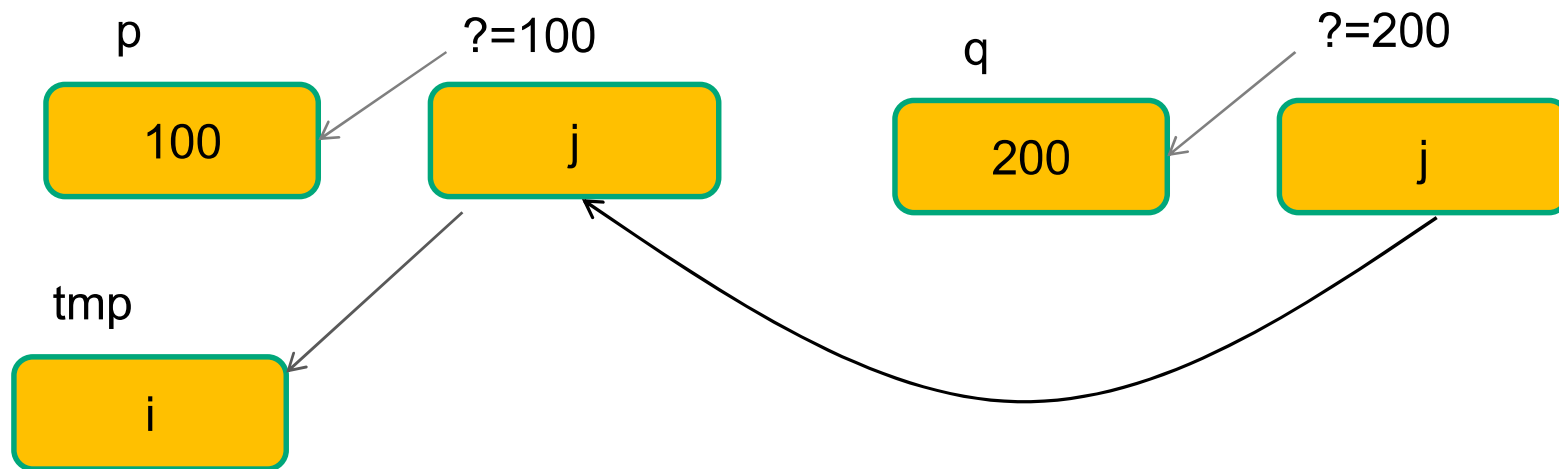
Call-by-Reference

```
void swap(int *p, int *q) {  
    int tmp;  
    tmp=*p; *p=*q; *q=tmp;  
}
```



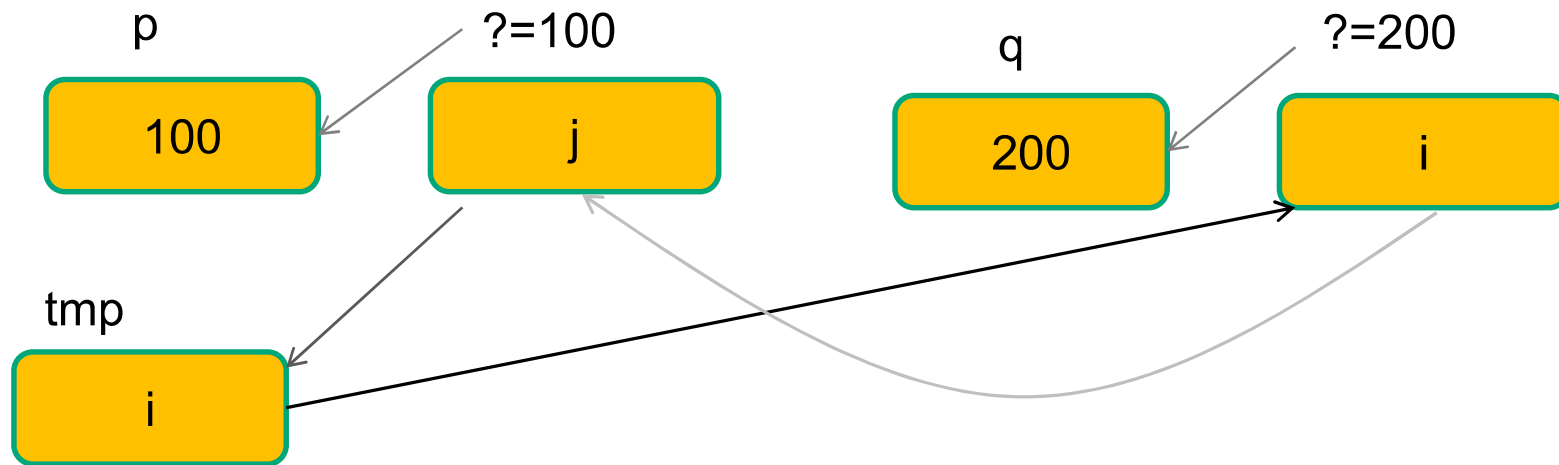
Call-by-Reference

```
void swap(int *p, int *q) {  
    int tmp;  
    tmp=*p; *p=*q; *q=tmp;  
}
```



Call-by-Reference

```
void swap(int *p, int *q) {  
    int tmp;  
    tmp=*p; *p=*q; *q=tmp;  
}
```



Call-by-Reference

```
void swap(int *p, int *q) {  
    int tmp;  
    tmp=*p; *p=*q; *q=tmp; }
```

1. Declare function parameter to be a pointer

2. Use dereferenced pointer in the function body

```
int main(void) {  
    int i = 3, j = 5;  
    swap(&i, &j);  
    printf("Value of i=%d,j=%d\n",i,j);}
```

3. Pass address as argument to the function call

p



i



q



j



Another example

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int Var = 10;
```

```
    int *ptr = &Var;
```

```
    printf("Address of Var = %p\n", ptr);
```

```
    printf("Stored value of Var = %d\n", *ptr);
```

```
    // We can also use ptr as lvalue
```

```
    *ptr = 20; // Value at address is now 20
```

```
    printf("*ptr is %d\n", *ptr);
```

```
    return 0;
```

```
}
```

Address of Var

Var

10

0x7fff98b499e8

ptr

0x7fff98b499e8

0x7fffa0757dd4

Address of ptr

Another example

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int Var = 10;
```

```
    int *ptr = &Var;
```

```
    printf("Address of Var = %p\n", ptr);
```

```
    printf("Stored value of Var = %d\n", *ptr);
```

```
    // We can also use ptr as lvalue
```

```
    *ptr = 20; // Value at address is now 20
```

```
    printf("*ptr is %d\n", *ptr);
```

```
    return 0;
```

```
}
```

Address of Var

Var

10

0x7fff98b499e8

ptr

0x7fff98b499e8

0x7fffa0757dd4

Address of ptr

Another example

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int Var = 10;
```

```
    int *ptr = &Var;
```

```
    printf("Address of Var = %p\n", ptr);
```

```
    printf("Stored value of Var = %d\n", *ptr);
```

```
    // We can also use ptr as lvalue
```

```
    *ptr = 20; // Value at address is now 20
```

```
    printf("*ptr is %d\n", *ptr);
```

```
    return 0;
```

```
}
```

Address of Var

Var

20

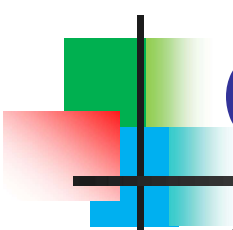
0x7fff98b499e8

ptr

0x7fff98b499e8

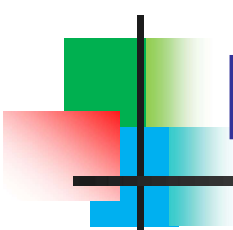
0x7fffa0757dd4

Address of ptr



Call-by-reference: benefits

- Efficiency: Passing pointer to a function is more efficient than passing a large data structure such as an array
- In-place modification: Function can modify the arguments directly, which may be helpful in certain cases
- Multiple return values: Pointer can be used to pass multiple values back from a function



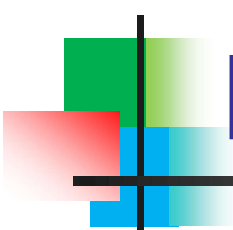
Ex:

- Write a C function to compute the area and perimeter of a triangle whose sides a, b, and c are given by the user as inputs.
- `void area_peri(float x, float y, float z, float *area, float *peri)`

Formula to compute perimeter = $a + b + c$

Formula to compute area = $\sqrt{s(s-a)(s-b)(s-c)}$

Where $s = 0.5 * (a+b+c)$



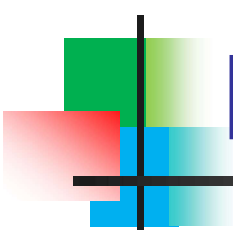
Pointer return values

- A function can also return pointer values

```
return-type * function-name(parameter list )  
{  
    type *ptr1;  
    // declarations and statements  
  
    return ptr;  
}
```

- Calling function

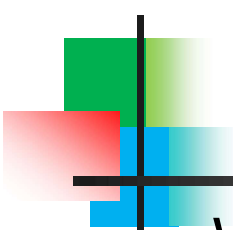
```
int main {  
// declarations and statements  
    type *ptr2;  
    ptr = function-name(argument list)  
  
}
```



Ex: Pointer return values

- Write a function to find the maximum in an array passed as argument.

```
float *findMax(float A[], int size)
```

- 
-
- Write a function to compute the cube of a number using pointers.
 - Write a function to compute the maximum of two numbers using pointers.